BÀI TẬP BT03

TÌM KIẾM, CSP VÀ GAME TRONG AI

Bài 1. Minimax và Alpha-Beta Pruning

a) Áp dụng Minimax, tính giá trị các nút trung gian

Nguyên tắc:

- Nút MAX: chọn giá trị lớn nhất trong các nút con.
- Nút MIN: chọn giá trị nhỏ nhất trong các nút con.
- Tính từ tầng lá lên nút gốc.

Tính 9 nút MAX ở tầng dưới:

Nút MAX	Giá trị lá	Giá trị nút
M1	3, 0	3
M2	-1, 1	1
M3	4, 6	6
M4	10, -1	10
M5	0, -2	0
M6	12, -3	12
M7	-4, -2	-2
M8	-2, 10	10
M9	-5, -7	-5

Tính 3 nút MIN:

- Nút MIN bên trái: min(3, 1, 6) = 1.
- Nút MIN ở giữa: min(10, 0, 12) = 0.
- Nút MIN bên phải: min(-2, 10, -5) = -5.

Tính nút gốc MAX:

$$V = \max(1, 0, -5) = 1$$

b) MAX nên chọn nhánh nào? Giá trị trò chơi

MAX chọn nhánh bên trái, vì nhánh này có giá trị Minimax bằng 1, lớn hơn 0 và -5. Giá trị trò chơi là 1.

c) Áp dụng Alpha-Beta Pruning

- α: giá trị tốt nhất MAX đã bảo đảm được.
- β: giá trị tốt nhất MIN đã bảo đảm được.
- Khi $\alpha \geq \beta$, có thể cắt tỉa các nhánh còn lại của nút đang xét.

Duyệt từ trái sang phải:

1. Nhánh trái: M1 = 3, cập nhật $\alpha = 3$. M2 = max(-1, 1) = 1, nút MIN bên trái cập nhật $\beta = 1$. Vì $\beta = 1 \leq \alpha = 3$, cắt tỉa M3, tức hai lá 4 và 6.
2. Nhánh giữa: M4 = 10; M5 = 0. Nút MIN ở giữa có $\beta = 0$. Vì $\beta = 0 \leq \alpha = 1$ tại gốc, cắt tỉa M6, tức hai lá 12 và -3.
3. Nhánh phải: M7 = max(-4, -2) = -2. Nút MIN bên phải có $\beta = -2$. Vì $\beta = -2 \leq \alpha = 1$, cắt tỉa M8 và M9, tức các lá (-2, 10) và (-5, -7).

d) Các nút lá không cần thăm

- Nhánh trái: 4, 6.
- Nhánh giữa: 12, -3.
- Nhánh phải: -2, 10, -5, -7.

Minimax đánh giá đủ 18 nút lá; Alpha-Beta đánh giá 10 nút lá. Alpha-Beta giảm 8 lần đánh giá nút lá mà vẫn giữ nguyên giá trị tối ưu 1.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Nút ngẫu nhiên C lấy giá trị kỳ vọng, không lấy giá trị lớn nhất hoặc nhỏ nhất:

$$V(C) = \Sigma\, p_i V_i$$

Với hai con có xác suất bằng nhau:

$$V(C) = (V_1 + V_2) / 2$$

a), b), c) Ba trường hợp trong Hình 2

Các ô lá được để trống để người làm tự gán giá trị. Đề cho phép chọn giá trị hữu hạn, nhưng không cho sẵn miền giá trị tiện ích. Vì vậy, điều kiện cắt tỉa cần được xét dựa trên giới hạn có thể có của các lá chưa duyệt.

- Trường hợp 1: Có thể cắt tỉa nếu giá trị của nút MAX bên phải đã được bảo đảm lớn hơn giá trị β của nút MIN gốc. Khi đó MIN sẽ không chọn nhánh này. Nếu các giá trị lá chưa duyệt không có giới hạn trên đã biết, chưa thể kết luận chỉ từ một lá đã xem.
- Trường hợp 2: Có thể cắt tỉa nếu giới hạn trên của giá trị kỳ vọng tại nút ngẫu nhiên bên phải không vượt quá giá trị tốt nhất hiện có của MIN gốc. Cần tính cả xác suất của các kết quả chưa duyệt và giới hạn giá trị của chúng.
- Trường hợp 3: Áp dụng nguyên tắc giới hạn kỳ vọng tương tự. Nếu chưa có giới hạn tiện ích cho lá chưa duyệt thì không đủ cơ sở để khẳng định có thể cắt tỉa.

Không cắt tỉa khi hai giá trị chỉ bằng nhau, đúng theo yêu cầu của đề.

d) Vì sao cắt tỉa ở cây ngẫu nhiên khác với Alpha-Beta thông thường?

Trong cây Minimax, các nút MAX và MIN dùng phép lấy cực đại hoặc cực tiểu. Khi một nhánh đã đủ để chứng minh rằng nút cha không thể cải thiện kết quả, các nhánh còn lại có thể được bỏ qua.

Trong cây Expinimax, nút ngẫu nhiên dùng trung bình có trọng số xác suất. Một kết quả chưa xét vẫn có thể làm thay đổi giá trị kỳ vọng. Do đó, chỉ có thể cắt tỉa khi đã có các giới hạn đủ chặt để chứng minh rằng các kết quả chưa xét không thể thay đổi quyết định ở nút cha.

Bài 3. Câu hỏi đúng/sai

a) Luôn có V_M ≤ V_E

Đáp án: Đúng.

Tại mỗi nút MIN, giá trị Minimax là giá trị nhỏ nhất trong các nút con. Khi thay nút MIN bằng nút ngẫu nhiên, giá trị mới là trung bình có trọng số của các nút con.

$$\min(V_1, \ldots, V_n) \leq \Sigma\, p_i V_i$$

với $p_i \geq 0$ và $\Sigma p_i = 1$. Vì phép lấy MAX không làm đảo ngược quan hệ này nên V_M ≤ V_E.

b) Chính sách Minimax tối ưu bảo đảm lợi ích kỳ vọng ít nhất là V_M

Đáp án: Đúng, với giả định cùng mô hình trạng thái, tiện ích và các lựa chọn hành động. Chính sách Minimax bảo đảm kết quả không thấp hơn giá trị Minimax khi đối thủ hành động tối ưu để giảm tiện ích. Khi thay đối thủ bằng nút ngẫu nhiên có phân phối xác suất đã biết, giá trị kỳ vọng tại nút đó không nhỏ hơn giá trị nhỏ nhất của các nút con.

c) Chính sách Minimax tối ưu bảo đảm lợi ích kỳ vọng ít nhất là V_E

Đáp án: Sai. Chính sách tối ưu theo Minimax không nhất thiết tối ưu theo Expectimax. Minimax chọn hành động dựa trên trường hợp đối thủ gây bất lợi nhất; Expectimax chọn hành động có giá trị kỳ vọng lớn nhất. Vì thế, chính sách Minimax chỉ được bảo đảm ít nhất V_M, không nhất thiết đạt V_E.

d) Trò chơi một người, kết quả ngẫu nhiên, thắng +1, thua -1

- Expectimax với tiện ích trạng thái cuối +1/-1: chọn.
- Value Iteration với phần thưởng trung gian 0 và thắng/thua +1/-1: chọn.
- Minimax: không phải lựa chọn phù hợp theo mô hình thông thường nếu không có đối thủ chủ động làm giảm tiện ích; có thể phù hợp khi có giả định bổ sung.

Expectimax phù hợp vì trò chơi có kết quả ngẫu nhiên; tác tử chọn hành động có lợi ích kỳ vọng cao nhất. Value Iteration phù hợp nếu trò chơi được mô hình hóa thành MDP với xác suất chuyển trạng thái, phần thưởng và điều kiện kết thúc được xác định.

Bài 4. CSP với Constraint Graph và lan truyền ràng buộc

a) Đồ thị ràng buộc

Các đỉnh: A, B, C, D, E, F.

Các cạnh: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

b) Sau khi gán A, Forward Checking có thể thay đổi miền của biến nào?

Đáp án: B, C, D. Forward Checking chỉ xem xét các biến chưa gán có cạnh nối trực tiếp với biến vừa gán. Do A nối trực tiếp với B, C và D, miền của ba biến này có thể bị thu hẹp.

c) Sau khi gán A, tiếp tục gán B: Forward Checking tại B thay đổi miền nào?

Đáp án: C, E. B nối trực tiếp với A, C và E. A đã được gán từ trước nên các biến chưa gán cần kiểm tra tại bước này là C và E.

d) Sau khi gán A, duy trì Arc Consistency: miền nào có thể thay đổi?

Đáp án: B, C, D, E, F, với giả định các cạnh biểu diễn ràng buộc hai biến phải khác giá trị nhau. A có thể làm thu hẹp B, C, D; miền B có thể làm thay đổi C và E; miền C có thể làm thay đổi F. Sự lan truyền có thể đi xa hơn các đỉnh kề trực tiếp của A.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency

Đáp án: C, E, F có thể bị thay đổi miền. B được gán nên miền C và E có thể bị thu hẹp; sự thay đổi miền C có thể lan truyền sang F. D không nối trực tiếp với B, C hoặc E nên không bị thay đổi do bước gán B nếu không có ràng buộc khác.

f), g), h) Số lần backtrack trong trường hợp xấu nhất

Chỉ có đồ thị và kích thước miền d thì chưa đủ để xác định số backtrack chính xác. Cần biết nội dung các ràng buộc, cách chọn giá trị trong miền và quy ước đếm backtrack. Nếu ràng buộc ban đầu không nhất quán, thuật toán có thể phát hiện vô nghiệm ngay trước khi gán biến.

- Nếu sau mỗi lần duy trì Arc Consistency, các miền đều còn giá trị và cấu trúc ràng buộc được bảo đảm nhất quán, số lần quay lui có thể giảm đáng kể.
- Nếu ràng buộc chỉ gây mâu thuẫn khi gán một biến ở sâu trong cây tìm kiếm, thuật toán có thể phải thử nhiều tổ hợp giá trị trước khi quay lui.
- Arc Consistency chỉ bảo đảm tính nhất quán của từng cung; nó không bảo đảm mọi CSP đều có nghiệm chỉ bằng việc kiểm tra cung.

Không nên tự đưa ra một số duy nhất khi đề chưa quy định các dữ kiện đó.

Bài 5. Cây trò chơi Pacman và Casper

a) Điền các cặp (u, v) còn thiếu

- u: giá trị Pacman tin tưởng khi nghĩ Casper là đối thủ muốn giảm tiện ích.
- v: giá trị thực tế khi Casper thân thiện và muốn tăng tiện ích cho Pacman.
- Nút Pacman: chọn nhánh có u lớn nhất; giá trị v đi theo nhánh được chọn.
- Nút Casper: u lấy giá trị nhỏ nhất của các con, còn v lấy giá trị lớn nhất của các con.

Tính các nút Casper ở tầng dưới:

$$(4, 6) = (\min(4, 6), \max(4, 6))$$

$$(1, 7) = (\min(1, 7), \max(1, 7))$$

$$(0, 6) = (\min(0, 6), \max(0, 6))$$

$$(3, 5) = (\min(3, 5), \max(3, 5))$$

Tính các nút Pacman ở tầng kế tiếp:

- Nhánh thứ nhất so sánh (2,2) với (1,7). Vì 2 > 1, kết quả là (2,2).
- Nhánh thứ hai so sánh (1,1) với (0,6) và (3,5) ở các nút con tương ứng. Nút Pacman chọn nhánh có u lớn nhất, tức (1,1).

Tính hai nút Casper bên trên:

- Nút Casper bên trái có các con (2,2) và (1,1), nên giá trị là (1,2).
- Nút Casper bên phải có các con (3,9) và (4,8), nên giá trị là (3,9) nếu xét phép MIN trên u và MAX trên v.

Tính nút gốc Pacman: so sánh (1,2) và (3,9), Pacman chọn nhánh phải vì 3 > 1. Giá trị gốc: (3,9).

b) Các nhánh có thể không cần duyệt

Duyệt từ trái sang phải, có thể bỏ qua các nhánh khi giá trị đã xét đủ để chứng minh chúng không thể thay đổi quyết định tối ưu. Việc xác định chính xác từng nhánh bị cắt tỉa cần xét cận α, β theo giá trị Pacman tin tưởng u, đồng thời bảo đảm việc chọn nhánh thực tế của Casper theo v không làm thay đổi quyết định. Không thể kết luận chỉ từ giá trị v mà bỏ qua u.

c) Nếu Pacman biết Casper thân thiện

Nếu Pacman biết Casper thực sự muốn tăng tiện ích cho mình, Pacman có thể lựa chọn hành động dựa trên hành vi thực tế của Casper thay vì giả định Casper là đối thủ. Bài toán chuyển từ mô hình Minimax sang mô hình có thông tin về mục tiêu của Casper. Giá trị tối ưu có thể tăng hoặc giữ nguyên; không nên mặc định mọi nút đều đổi sang MAX nếu chưa xác định chính xác thứ tự hành động và các nhánh trong cây.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Thuật toán	Điều kiện đầy đủ (Complete)	Điều kiện tối ưu (Optimal)
BFS	Đầy đủ nếu hệ số phân nhánh hữu hạn và tồn tại lời giải ở độ sâu hữu hạn.	Tối ưu khi mọi bước có cùng chi phí.
DFS	Không bảo đảm trong cây có độ sâu vô hạn hoặc có chu trình nếu không kiểm soát. Bảo đảm trong không gian hữu hạn khi xử lý chu trình phù hợp.	Không bảo đảm tối ưu.
IDS	Đầy đủ khi hệ số phân nhánh hữu hạn và lời giải ở độ sâu hữu hạn.	Tối ưu khi mọi bước có cùng chi phí.
A*	Đầy đủ nếu hệ số phân nhánh hữu hạn, chi phí bước dương	Tối ưu với heuristic chấp nhận được trong tree search; với

	được chặn dưới bởi một số dương và các điều kiện tìm kiếm phù hợp được thỏa mãn.	graph search, heuristic nhất quán bảo đảm tối ưu khi đóng nút theo cách chuẩn.

Giải thích về chi phí bước và heuristic

- BFS tìm lời giải nông nhất, không nhất thiết tìm lời giải có tổng chi phí thấp nhất nếu chi phí các bước khác nhau.
- DFS có thể đi sâu vào một nhánh dài mà bỏ qua lời giải tốt hơn ở nhánh khác.
- IDS kết hợp ưu điểm về tính đầy đủ của BFS với bộ nhớ thấp của DFS, nhưng tối ưu chi phí khi các bước có cùng chi phí.
- A* sử dụng $f(n) = g(n) + h(n)$, trong đó $g(n)$ là chi phí từ đầu đến nút hiện tại và $h(n)$ là ước lượng chi phí còn lại. Heuristic cần thỏa điều kiện phù hợp để bảo đảm tối ưu.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* đã tìm được đường tối ưu, chi phí cạnh $X \rightarrow Y$ thay đổi. Không nhất thiết phải chạy lại từ đầu, nhưng cần cập nhật các giá trị chi phí bị ảnh hưởng và kiểm tra xem đường cũ còn tối ưu hay không.

a) Chi phí tăng n, cạnh thuộc đường tối ưu cũ

Chi phí đường cũ tăng thêm n. Đường cũ có thể không còn tối ưu vì có thể tồn tại đường khác rẻ hơn. Cần cập nhật chi phí đường cũ và kiểm tra các đường thay thế. Nếu không có đường nào tốt hơn, đường cũ vẫn là tối ưu.

b) Chi phí giảm n, cạnh thuộc đường tối ưu cũ

Chi phí đường cũ giảm n, nên đường cũ vẫn tối ưu nếu mọi đường khác không giảm theo cách làm chúng tốt hơn. Vì chỉ một cạnh trên đường tối ưu cũ giảm chi phí, chi phí tối ưu mới bằng chi phí cũ trừ n, với giả định cạnh mới hợp lệ.

c) Chi phí tăng n, cạnh không thuộc đường tối ưu cũ nhưng đã được khám phá

Chi phí của cạnh này tăng nhưng đường tối ưu cũ không sử dụng cạnh đó. Do đó, chi phí tối ưu không đổi và đường cũ vẫn tối ưu. Có thể cập nhật thông tin cạnh để phục vụ các truy vấn tiếp theo.

d) Chi phí giảm n, cạnh không thuộc đường tối ưu cũ nhưng đã được khám phá

Đường cũ vẫn có chi phí như trước, nhưng cạnh mới rẻ hơn có thể tạo ra một đường tốt hơn. Cần cập nhật chi phí từ X đến Y, sau đó kiểm tra các đường từ Y đến đích. Chi phí tối ưu có thể giảm hoặc giữ nguyên.

e) Chi phí tăng n, cạnh không thuộc đường tối ưu cũ và chưa được khám phá

Đường tối ưu cũ không sử dụng cạnh này, nên chi phí tối ưu đã biết không đổi. Tuy nhiên, vì cạnh chưa được khám phá, thông tin hiện tại không đủ để kết luận về mọi đường đi mới trong mô hình mà đồ thị chưa được biết đầy đủ. Nếu đồ thị và các cạnh đã được biết đầy đủ từ đầu, việc tăng chi phí của một cạnh không thuộc đường tối ưu cũ không làm thay đổi giá trị tối ưu.

f) Chi phí giảm n, cạnh không thuộc đường tối ưu cũ và chưa được khám phá

Cạnh giảm chi phí có thể tạo ra một đường đi tốt hơn đường cũ. Nếu cạnh này được khám phá, cần cập nhật g(Y), cập nhật hàng đợi ưu tiên và tiếp tục A* từ những nút bị ảnh hưởng. Không nhất thiết phải khởi động lại toàn bộ thuật toán nếu còn lưu được các giá trị chi phí và thông tin tiền nhiệm.

Bài 8. Tác tử thông minh và môi trường

- A: act(percept) → f_A(); hàm không nhận percept nên không thay đổi hành động theo quan sát.
- B: act(percept) → f_B(percept); dùng percept hiện tại để quyết định hành động.
- C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts), nên sử dụng lịch sử quan sát.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh

Đáp án: B và C. Môi trường quan sát đầy đủ, xác định và tĩnh cho phép tác tử dựa trên trạng thái quan sát hiện tại để lựa chọn hành động hợp lý. B dùng percept hiện tại; C có thể dùng lịch sử percept. A không thể bảo đảm hành động hợp lý trong mọi môi trường thuộc lớp này vì không sử dụng thông tin quan sát.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh

Đáp án: C. Do chỉ quan sát được một phần trạng thái, percept hiện tại có thể chưa đủ để xác định hành động tối ưu. Lịch sử percept giúp tác tử suy luận trạng thái ẩn từ các quan sát trước đó.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh

Đáp án: C. Trong môi trường vừa quan sát một phần vừa ngẫu nhiên, tác tử cần sử dụng lịch sử quan sát để ước lượng trạng thái hiện tại và đánh giá xác suất của các kết quả. Tác tử C có khả năng lưu lịch sử nên phù hợp nhất trong ba loại.

d) Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý?

Đáp án: Đúng. Ví dụ, xét môi trường chỉ có một hành động khả thi tại mỗi trạng thái. Khi đó, bất kể tác tử chọn hành động bằng cách nào, nó đều phải thực hiện hành động duy nhất đó. Vì không có lựa chọn khác tốt hơn, hành động của mọi tác tử đều hợp lý trong môi trường này.